МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №5

з курсу: „Паралельні обчислення”
тема: “Розробка HTTP-сервера для хостингу статичного веб-сайту.”

виконав студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович

КИЇВ
2024

Мета: Вивчити принцип роботи http веб-серверів. Створити простий веб-сервер для багатопотокової обробки GET-запитів. Засвоїти інструментарій для навантажувального тестування.

Хід роботи

1. Спроектувати веб сервер для хостингу статичного контенту. Можна обирати довільну технологією, вимогою є програмування на транспортному (четвертому) рівні стеку OSI (з допомогою сокетів).

2. Веб-сервер повинен опрацьовувати http запити з багатьох клієнтів. Реалізуємо GET відповідно до протоколу HTTP 1.1. RFC9112 RFC9110
Root-запит веде на index.html (приклади запитів “GET http://localhost:8080”, “GET http://localhost:8080/second_page.html”).

3. Захостити щонайменше 2 веб-сторінки (текст файлів наведено в Додатку 1, або можете додати власні). При цьому повинні оброблятися неіснуючі сторінки. (404 відповідь)

4. Провести навантажувальне тестування з допомогою сервісів jMeter або Locust. Визначити точку відмови в кількості паралельних клієнтів.

5. Зробити висновки, припустити яким чином можна зробити сервер більш ефективним і підвищити його здатність тримати навантаження.

6. (*) Додаткове завдання на плюс 1 бал – оптимізувати веб-сервер, зробивши його неблокуючим.

HTTP (англ. HyperText Transfer Protocol — «протокол передачі гіпертексту») — протокол прикладного рівня передачі (спочатку — як гіпертекстових документів). Основою HTTP є технологія «клієнт-сервер», тобто передбачається існування клієнтів, які ініціюють з'єднання та надсилають запит, і серверів, які очікують на з'єднання для отримання запиту, роблять необхідні дії та повертають назад повідомлення з результатом. Порт: 80/8080.
Основним об'єктом маніпуляції в HTTP є ресурс, який вказує URI (англ. Uniform Resource Identifier) у запиті клієнта. Зазвичай такими ресурсами є файли, що зберігаються на сервері, але ними можуть бути логічні об'єкти або щось абстрактне.

Для виконання даної роботи було написано код для серверу та для клієнта.
Коротке пояснення:
Сервер - Використовує багатопоточність для одночасного обслуговування множини запитів. Коли сервер отримує запит, він алокує задачу у ThreadPool для обробки запиту без блокування головного потоку, що слухає на порту.

Клієнт - Створює множину потоків, де кожен потік симулює окремого клієнта, який відправляє запит до сервера. Кожен потік створює свій сокет і відправляє запит, отримує відповідь, і закриває сокет.

Код сервера та клієнта взаємопов'язані через мережеві запити і відповіді. Сервер спроможний обслуговувати багато клієнтів одночасно, тоді як клієнти випробовують сервер, відправляючи запити одночасно.

Код серверу

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

const int PORT = 8080;
const std::string DIRECTORY_PATH =
"C:/Users/user/CLionProjects/Paralel_5_Server1/cmake-build-debug/";

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queue_mutex;
    std::condition_variable con_var;
    bool stop;

public:
    ThreadPool(size_t threads) : stop(false) {
        for (size_t i = 0; i < threads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(this->queue_mutex);
                        this->con_var.wait(lock, [this] { return this->stop || !this-
>tasks.empty(); });
                        if (this->stop && this->tasks.empty())
                            return;
                        task = std::move(this->tasks.front());
                        this->tasks.pop();
                    }
                    task();
                }
            });
        }
    }
    void enqueue(std::function<void()> task) {
        {
            std::unique_lock<std::mutex> lock(queue_mutex);
            if (stop) {
                std::cout << "Attempt to enqueue on stopped ThreadPool." << std::endl;
                return;
            }
            tasks.emplace(task);
        }
        con_var.notify_one();
```

```
    }
    ~ThreadPool() {
        {
            std::unique_lock<std::mutex> lock(queue_mutex);
            stop = true;
        }
        con_var.notify_all();
        for (std::thread &worker : workers) {
            worker.join();
        }
    }
};

std::string Read_file(const std::string& filePath) {
    std::ifstream file(DIRECTORY_PATH + filePath);
    if (!file.is_open()) {
        return "";
    }
    std::stringstream buff;
    buff << file.rdbuf();
    return buff.str();
}

void handleRequest(SOCKET clientSocket) {
    char buffer[1024];
    int bytesReceived = recv(clientSocket, buffer, sizeof(buffer), 0);
    if (bytesReceived > 0) {
        buffer[bytesReceived] = '\0';
        std::string request(buffer);
        std::size_t pos = request.find("GET ");
        if (pos != std::string::npos) {
            std::size_t endPos = request.find(" HTTP/1.1", pos + 4);
            if (endPos != std::string::npos) {
                std::string path = request.substr(pos + 4, endPos - (pos + 4));
                std::string filename;
                if (path == "/page2.html") {
                    filename = "page2.html";
                }
                else {
                    filename = "index.html";
                }

                std::string content = Read_file(filename);
                std::string response;
                if (content.empty()) {
                    std::string notFoundMessage = "<html><body><h1>404 Not
Found</h1></body></html>";
                    response = "HTTP/1.1 404 Not Found\r\nContent-Length: " +
std::to_string(notFoundMessage.length()) + "\r\nContent-Type: text/html\r\n\r\n" +
notFoundMessage;
                }
                else {
                    response = "HTTP/1.1 200 OK\r\nContent-Length: " +
std::to_string(content.length()) + "\r\nContent-Type: text/html\r\n\r\n" + content;
                }
                send(clientSocket, response.c_str(), response.length(), 0);
            }
        }
    }
    closesocket(clientSocket);
}

void startServer() {
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        std::cerr << "WSAStartup failed.\n";
        exit(1);
    }

    SOCKET serverSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (serverSocket == INVALID_SOCKET) {
```

```
        std::cerr << "Socket creation failed: " << WSAGetLastError() << '\n';
        WSACleanup();
        exit(1);
    }

    sockaddr_in serverAddr;
    memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = htonl(INADDR_ANY);
    serverAddr.sin_port = htons(PORT);

    if (bind(serverSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
        std::cerr << "Bind failed: " << WSAGetLastError() << '\n';
        closesocket(serverSocket);
        WSACleanup();
        exit(1);
    }

    if (listen(serverSocket, SOMAXCONN) == SOCKET_ERROR) {
        std::cerr << "Listen failed: " << WSAGetLastError() << '\n';
        closesocket(serverSocket);
        WSACleanup();
        exit(1);
    }

    std::cout << "Server is listening on port " << PORT << "...\n";
    ThreadPool pool(4);

    while (true) {
        sockaddr_in clientAddr;
        int clientAddrSize = sizeof(clientAddr);
        SOCKET clientSocket = accept(serverSocket, (SOCKADDR*)&clientAddr,
&clientAddrSize);
        if (clientSocket != INVALID_SOCKET) {
            pool.enqueue([clientSocket]() { handleRequest(clientSocket); });
        }
    }
    closesocket(serverSocket);
    WSACleanup();
}

int main() {
    startServer();
    return 0;
}
```

Результат роботи коду:

```
Server is listening on port 8080...
```

Тобто сервер в мене ініціалізує бібліотеку Winsock для роботи з мережею. Створює сокет, налаштовує його на прослуховування на певному порті і приймає вхідні з'єднання. Використовує ThreadPool для асинхронної обробки вхідних HTTP запитів, що дозволяє одночасно обробляти кілька запитів. Функція handleRequest читає HTTP запит, визначає запитувану сторінку, генерує відповідь (HTML-сторінку або помилку 404) і відсилає її клієнт

Код клієнта:

```
#include <iostream>
#include <string>
```

```cpp
#include <vector>
#include <thread>
#include <chrono>
#include <mutex>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib")

const int PORT = 8080;
const std::string SERVER_IP = "127.0.0.1";

void initializeWinsock() {
    WSADATA wsaData;
    int result = WSAStartup(MAKEWORD(2, 2), &wsaData);
    if (result != 0) {
        std::cerr << "WSAStartup failed: " << result << std::endl;
        exit(1);
    }
}

SOCKET createSocket() {
    SOCKET sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == INVALID_SOCKET) {
        std::cerr << "Error at socket(): " << WSAGetLastError() << std::endl;
        WSACleanup();
        exit(1);
    }

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP.c_str(), &serverAddr.sin_addr);

    if (connect(sock, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
        std::cerr << "Connection failed: " << WSAGetLastError() << std::endl;
        closesocket(sock);
        WSACleanup();
        exit(1);
    }

    return sock;
}

void sendRequest(SOCKET sock, const std::string& path) {
    std::string request = "GET " + path + " HTTP/1.1\r\nHost: " + SERVER_IP + "\r\
nConnection: close\r\n\r\n";
    send(sock, request.c_str(), request.length(), 0);

    char buffer[4096];
    int received = recv(sock, buffer, sizeof(buffer), 0);
    if (received > 0) {
        std::cout << "Received response for " << path << ": " << std::string(buffer,
received) << std::endl;
    }

    closesocket(sock);
}

int main() {
    initializeWinsock();

    int numClients;
    std::cout << "Enter the number of clients: ";
    std::cin >> numClients;

    std::vector<std::thread> clients;
    for (int i = 0; i < numClients; ++i) {
        clients.push_back(std::thread([] {
            SOCKET sock = createSocket();
```

```
            sendRequest(sock, "/index.html");
            SOCKET sock2 = createSocket();
            sendRequest(sock2, "/page2.html");
        }));
    }
    for (auto& client : clients) {
        client.join();
    }
    WSACleanup();
    return 0;
}
```

Результат роботи коду:
index.html

```
Enter the number of clients: 1
Received response for /index.html: HTTP/1.1 200 OK
Content-Length: 293
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page 1</title>
</head>
<body>
    <h1>Welcome to Page 1</h1>
    <p>Click <a href="page2.html">here</a> to go to Page 2.</p>
</body>
</html>
```

page2.html

```
Received response for /page2.html: HTTP/1.1 200 OK
Content-Length: 298
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page 2</title>
</head>
<body>
    <h1>Welcome to Page 2</h1>
    <p>Click <a href="index.html">here</a> to go back to Page 1.</p>
</body>
</html>
```

У разі неіснуючої сторінки у коді прописано:

```
if (content.empty()) {
    std::string notFoundMessage = "<html><body><h1>404 Not Found</h1></body></html>";
    response = "HTTP/1.1 404 Not Found\r\nContent-Length: " + std::to_string( val: notFoundMessage.length()) + "\r\nContent-Type: text/html\r\n\r\n" + notFoundMessage;
}
```

І маємо :

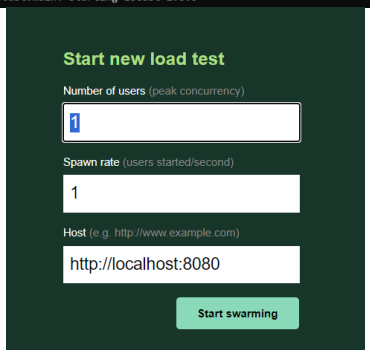
```
Enter the number of clients: 1
Received response for /index.html: HTTP/1.1 404 Not Found
Content-Length: 48
Content-Type: text/html

<html><body><h1>404 Not Found</h1></body></html>
```

Тобто клієнт також ініціалізує Winsock, створює сокет і з'єднується з сервером. Відправляє HTTP запити (GET) для вказаних URL-адрес (index.html і page2.html). Отримує відповіді від сервера і виводить їх.

Виконаємо тестування за допомогою програми locus, для цього виконаємо наступні дії.

```
D:\>locust -f locust.py
[2024-05-12 21:00:02,954] WIN-K0OKFI2NVR6/INFO/locust.main: Starting web interface at http://0.0.0.0:8089 (accepting connections from all network interfaces)
[2024-05-12 21:00:02,967] WIN-K0OKFI2NVR6/INFO/locust.main: Starting Locust 2.8.6
```



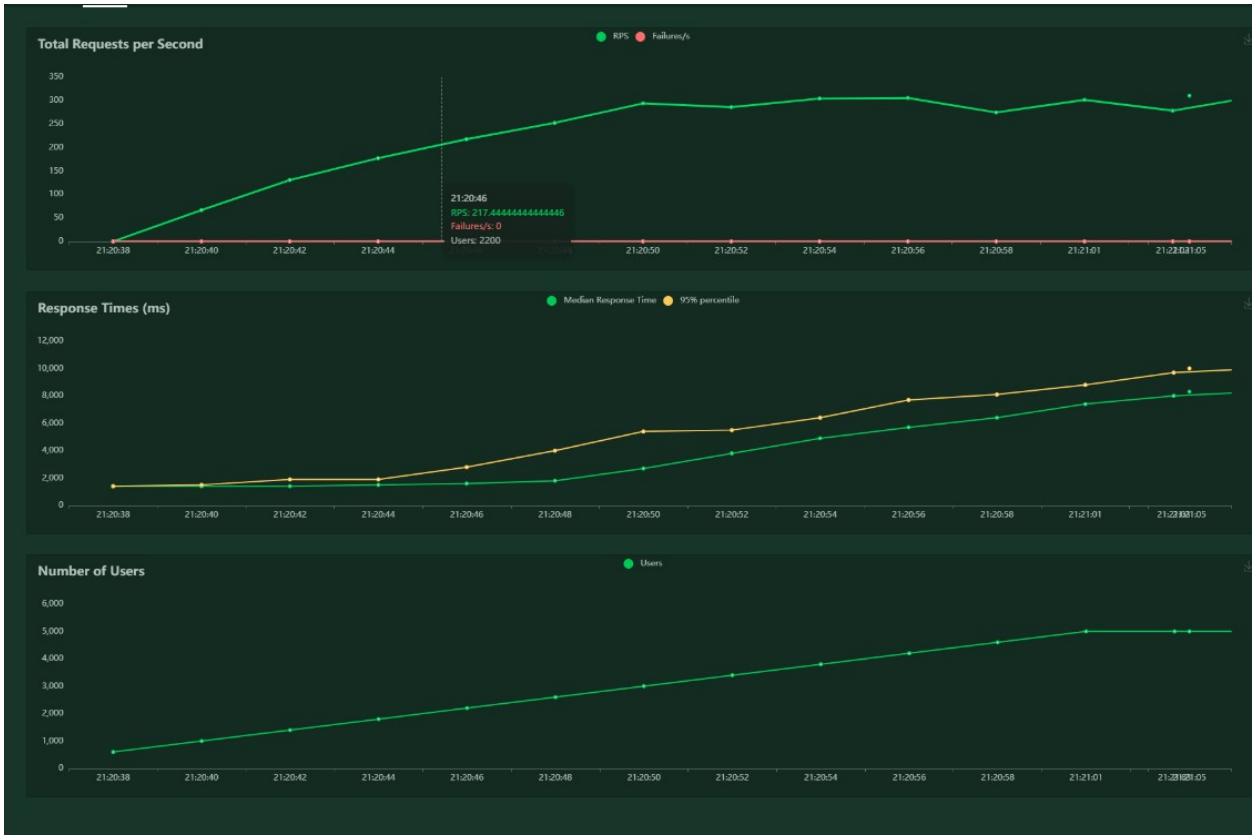
Зробимо тестування на 1000 клієнтів, 5000 та 10000 клієнтів
1000 клієнтів

Type	Name	# Requests	# Fails	Median (ms)	90%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/index.html	8692	0	1002	1200	1300	1068	1002	1615	293	204.6	0
GET	/page2.html	5233	0	1003	1200	1300	1069	1003	1612	298	123.4	0
	Aggregated	13925	0	1002	1200	1300	1068	1002	1615	295	328	0

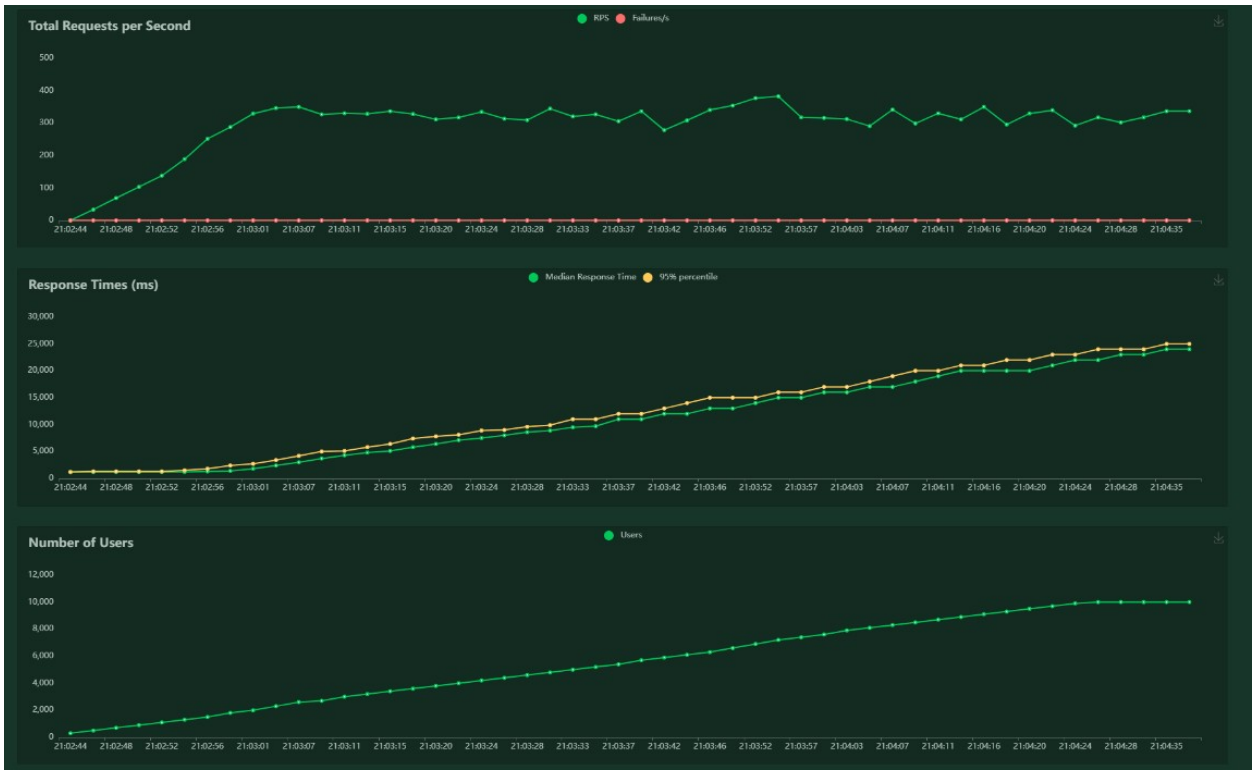


4 потоки + 5000 клієнтів.

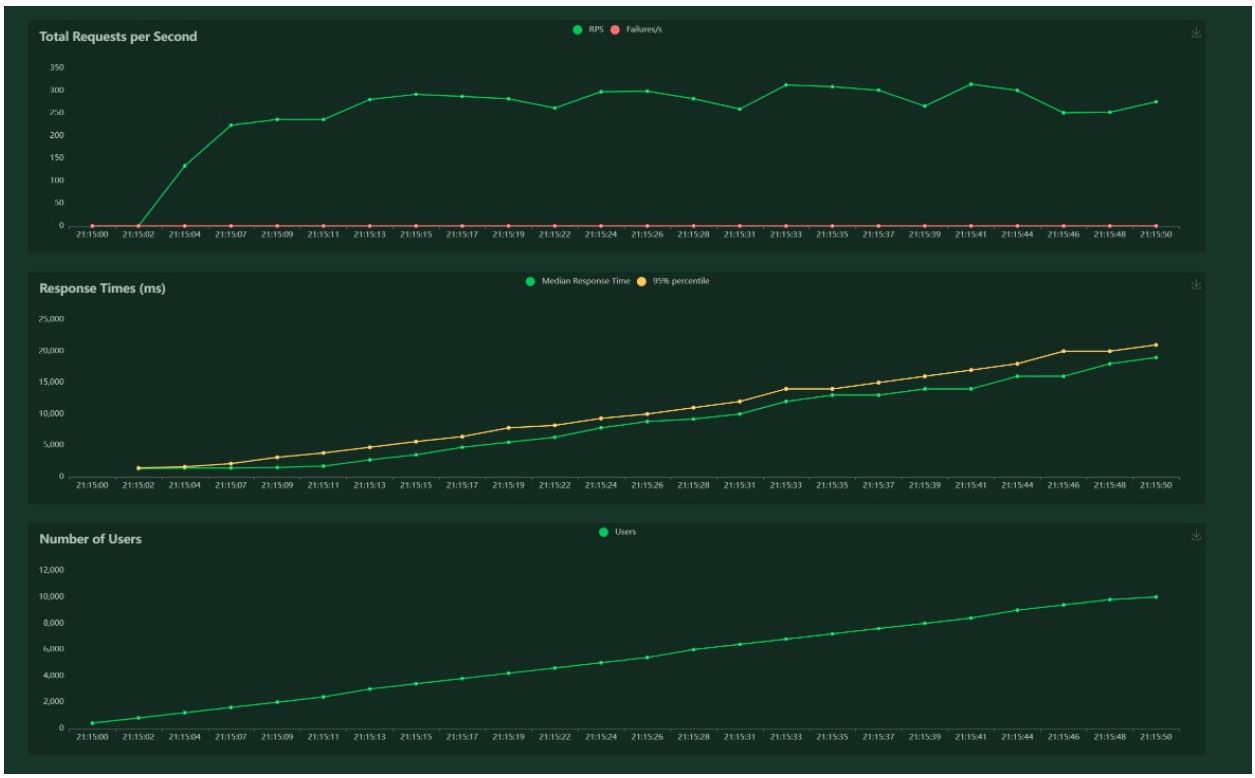
Type	Name	# Requests	# Fails	Median (ms)	90%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/index.html	5003	0	5200	9400	10000	5096	1003	10404	293	195	0
GET	/page2.html	3043	0	5100	9400	10000	5040	1005	10438	298	115.3	0
	Aggregated	**8046**	**0**	**5100**	**9400**	**10000**	**5075**	**1003**	**10438**	**295**	**310.3**	**0**



4 потоки + 10000 клієнтів



8 потоків + 10000 клієнтів.



Висновок:

У ході виконання лабораторної роботи №5 було реалізовано і протестовано веб сервер, на мові C++, сервер обробляв http запити методом GET з багатьох клієнтів одночасно. Було захощено веб-сторінки index.html" та "page2.html, а також додано обробку неіснуючих сторінок з відправленням відповіді з кодом статусу 404. Сервер було протестовано за допомогою locust, де ми навантажували наш сервер клієнтами.